

## Práctica 5: Esteganografía



Criptografía y Seguridad  
Universidad Nacional Autónoma de México  
Facultad de Ciencias

Anayanzi Delia Martínez Hernández  
Cecilia del Carmen Villatoro Ramos



9 de abril del 2025

*Esta práctica fue posible gracias a David Armando Silva de Paz.*

## Introducción

A pesar de que Estenografía es una palabra muy seria. Es probable que ya lo hayan hecho. ¿Alguna vez has usado jugo de limón como tinta? En una hoja de papel, lo escrito con jugo de limón es prácticamente invisible hasta que se usa calor para revelar lo escrito (Video). Este pequeño juego de niños es estenografía porque *ocultamos* un mensaje y solo es visible en condiciones adecuadas.

En la Segunda Guerra Mundial también se utilizaron otras técnicas. Por ejemplo, tomemos el siguiente mensaje aparentemente inofensivo:

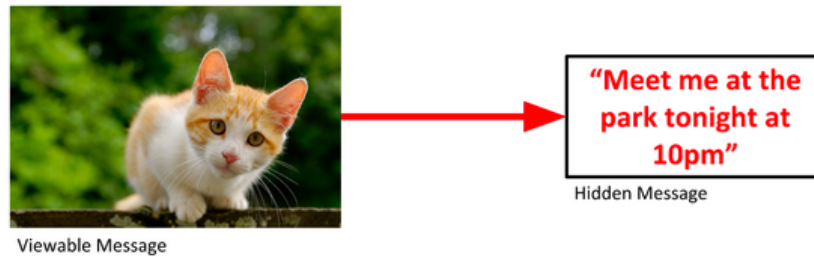
Apparently neutral's protest is thor-  
oughly discounted and ignored. Isman  
hard hit. Blockade issue affects pretext  
for embargo on by-products, ejecting  
suets and vegetable oils.

Si agarramos la segunda letra de cada palabra, obtenemos un mensaje que puede ser más peligroso.

Pershing sails from NY June 1 (one).

En nuestro siglo, durante el 2005 la *Electronic Frontier Foundation* publicó un artículo que nos indica que algunas impresoras de color de marcas como Xenrox y Hewlett-Packard también usan estenografía. Los documentos impresos en estas máquinas tienen pequeños puntos amarillos (imágenes) que guardan los números de serie de las impresoras y marcas de fecha y hora. La manera de leer estos datos fue dada por estas empresas al servicio secreto de Estados Unidos.

Actualmente podemos servirnos de la digitalización de la información para ocultar mensajes en archivos con muchos bits, como son las imágenes. En esta práctica ocultaremos mensajes usando los colores de las imágenes.



## Desarrollo

Como sabemos, la información en una computadora se guarda en sistema binario. Es decir, cada dato es una cadena de 0 y 1 (*bits*). Estas cadenas podemos dividirlos en bloques de ocho dígitos (*bytes*) y cambiar algunos *bits* no muy importantes para esconder nuestro mensaje entre ellos.

## Preámbulo

La técnica conocida como **LSB** es parecida a lo que se hizo en la Segunda Guerra Mundial. Pero en lugar de palabras tenemos *bytes* y en lugar de tomar una letra, tomaremos un *bit*. Este bit lo cambiaremos de acuerdo con nuestro mensaje.

Para esto usaremos una imagen y un mensaje. El mensaje lo sacaremos de **fortune**:

```
$ sudo apt install fortune
$ fortune
You have the capacity to learn from mistakes. You'll learn a lot today.
```

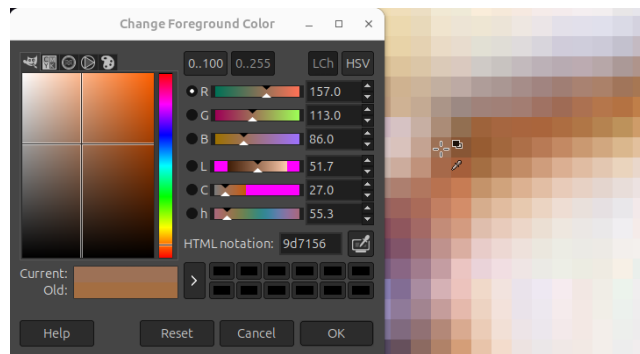
Usando un script en python, podemos sacar el valor binario de cada una de las letras de nuestro mensaje usando las funciones `bin`, `chr`, `ord`. Al final tenemos lo que lee la computadora. Obviamente los lee por hexadecimal. Es importante que estén en bloques de 8 para poder recuperar el mensaje de vuelta.

```
Y -> 89      01011001
o -> 111     01101111
u -> 117     01110101
      -> 32    00100000
h -> 104     01101000
a -> 97      01100001
v -> 118     01110110
e -> 101     01100101
...
01011001 01101111 01110101 00100000 01101000 01100001 01110110 01100101 ...
```

Y suponemos que la imagen es esta:



Hablando un poco de *Procesamiento Digital de imágenes PDI*, la imagen es una *matriz* de pixeles de tamaño  $n * m$ . Cada pixel está conformado por los componentes **Red**, **Green**, **Blue**, los canales de luz. Si abrimos la imagen con un editor de imágenes como *Gimp* y hacemos zoom podemos ver tal cual estos valores en número.



Que a su vez, también los podemos representar en binario, por ejemplo, para ese pixel son:

```
R: 157    10011101
G: 113    01110001
B: 86     01010110
... otro pixel
R: 158    10011110
```

G: 114    01110010  
B: 85    01010101  
... etc

Aquí está el espacio para poder acomodar nuestro mensaje, solo hay que acomodar con cuidado.

Tomemos la primera letra de nuestro mensaje Y -> 89 01011001. No podemos sustituir R: 157: 10011101 por R: 89: 01011001 porque cambia la intensidad de color a la mitad, lo cual ya es notable para el ojo humano, pero, podemos cambiar de R: 157: 10011101 a R: 156: 10011100 sustituyendo el último bit de 1 a 0, ya que 0 es el bit con el que empieza Y -> 89: 01011001.

El siguiente canal será G: 113: 01110001. Lo dejamos tal cual ya que el segundo bit de Y -> 89: 01011001 es 1 y coincide con el **LSB** de 113: 01110001.

El siguiente canal es B: 86: 01010110, no lo modificamos porque el tercer bit de Y -> 89: 01011001 es 0 y coincide con el último bit de 86: 01010110.

Pero para el siguiente pixel, con el canal R: 158: 10011110, sí necesitamos cambiar su último bit 0 por 1 ya que el cuarto bit de 89: 01011001 es 1.

Siguiendo con el algoritmo, podemos usar la última columna de los colores (el último bit) para poder meter un mensaje, que no es visible a simple vista, ya que de un valor R: 157 a R: 156 es imperceptible. ¿Puedes notar la diferencia entre estos 2 rojos?



**Nota:** La extensión .jpg utiliza un algoritmo de compresión para poder ahorrar espacio, modificando los valores de la imagen. A diferencia de la extensión .png que guarda **tal cual** los valores de cada pixel (ejemplo).

## Parte 1

Deberán crear un programa que inserte un mensaje oculto dentro de una imagen utilizando la técnica del **LSB**. Pueden desarrollarlo en su lenguaje de programación de preferencia. Deberán seguir estos lineamientos:

1. Para ocultar:

- Los primeros 2 bytes representarán la longitud del mensaje. No son dígitos, es el valor.  $[0, 2^{16} - 1]$
- Inmediatamente después, el mensaje a ocultar.
- Terminar con el carácter 'x00' (o byte 00000000) el mensaje.

2. Para revelar:

- Obtener la longitud del mensaje.
- Rescatar solamente el mensaje de longitud especificada.
- Si no se termina con carácter 'x00' en el fin de la longitud, abortar.
- Imprimir en salida estándar el mensaje oculto.

El programa deberá ser llamado con banderas `-h` y `-r` para *ocultar* y *revelar* respectivamente. Además, se deberá leer el mensaje por argumentos o por entrada estándar. La ruta de la imagen debe ser específicamente usando la bandera `-i`.

## Parte 2

Comprueba que tu programa es correcto, úsalo para revelar el mensaje oculto en la imagen que escogieron. Escriban el mensaje oculto en su pdf.

## Parte 3

Usa los programas vistos en clase `exiftool`, `steghide`, `xxd`, `strings`.

1. Encuentren la frase oculta con `steghide` en la imagen `practica_5.jpg`.
2. Cambien el comentario de la imagen por el nombre del equipo o su nombre propio (si trabajan solos).

## Preguntas

Contesta las siguientes preguntas con tus propias palabras. Serán sometidas a la prueba turing.

1. ¿Por qué la esteganografía forma parte de la criptografía? ¿Cuál es el elemento en común? Solo hay una respuesta correcta.
2. ¿Es detectable el **LSB** por los algoritmos de mensajería como *Whatsapp* o *Instagram*?
3. ¿En qué tipos de archivo **no** podemos usar LSB?
4. Teniendo una imagen de  $n * m$  píxeles, ¿cuál es la longitud máxima de mensaje (caracteres) que podemos incluir usando el último bit?
5. ¿A cuántos bits de diferencia podemos diferenciar colores con el ojo humano?

## Entrega

Para esta entrega, harán un PDF documentando el desarrollo y el/los archivo/s compresos en una carpeta `src/`. Basta con que un integrante del equipo lo suba, los nombres indicados en el pdf serán contados como que trabajaron y se les dará la misma calificación. Además el PDF deberá contener:

- **Nombres de los integrantes:** Portada del equipo, con nombres y números de cuenta, además del nombre del equipo.
- **Desarrollo:** Explicar de manera rápida cómo es que funciona su código, así como las instrucciones para poder ejecutar su programa y alguna ejecución del mismo. También se contestan aquí las preguntas.
- **Preguntas:** Respuestas a las preguntas de ésta práctica.
- **Conclusiones:** Comentar qué tan *peligroso/no peligroso* es la esteganografía, y si es buena idea usarla como algún *cifrado*. También pueden poner comentarios de cosas nuevas que aprendieron, trabajo en equipo, retroalimentación de la práctica, etc.
- **Referencias:** En caso de utilizar *bibliografía* de otro lado, favor de copiar el link y al autor en caso de ser posible. Hay que dar créditos al autor siempre.

## Referencias

1. Neil F. Johnson, Sushil Jajodia, George Mason University, *Exploring Steganography: Seeing the Unseen*, IEEE Computers, February 1998, pp. 26-34 <https://sci-hub.se/10.1109/mc.1998.4655281>
2. Vasleiadis C. Medium (2024).Steganography: Hide messages in image and audio files
3. Wolfshirtz, Cryptokait (2021). Why Steganography is Cryptography
4. Schoen, S. (2005, Octubre 16) *Secret code in color printers lets government track you*. Electronic Frontier Foundation. Consultado el 07 de abril del 2025.